

Documentación de la arquitectura y modelo de datos

Infraestructura y arquitectura para Big Data

Dato	Información
Estudiante	Sara Colorado Gil
Docente	César Luis Vásquez
Curso	PREICA2602B010136
Institución	Institución Universitaria Digital de Antioquia
Fecha	29 de septiembre de 2026

Arquitectura reproducible, trazable y preparada para evolucionar hacia servicios de nube.

1 Introducción

Este documento integra las tres primeras actividades del proyecto: ingesta desde un API, preprocesamiento de los registros y enriquecimiento con fuentes complementarias. La solución utiliza un repositorio de GitHub como espacio central de trabajo, SQLite como almacenamiento analítico local, Pandas para transformar los datos y GitHub Actions para ejecutar los procesos de manera reproducible.

La arquitectura conserva 194 productos durante todo el flujo. La EA2 corrige 92 marcas faltantes, deja cero valores nulos y señala 12 precios atípicos. La EA3 integra seis fuentes con 194 coincidencias por cada una, amplía el dataset de 14 a 23 campos y genera un archivo final listo para análisis.

1.1 Alcance

- Describir los componentes y el flujo de datos implementado entre EA1, EA2 y EA3.
- Documentar el esquema físico de SQLite y el modelo lógico utilizado para la integración.
- Justificar SQLite, Pandas, PySpark y GitHub Actions según el tamaño y evolución esperada del proyecto.
- Explicar cómo el repositorio y los workflows simulan servicios de almacenamiento y procesamiento en la nube.

1.2 Resultado principal

El resultado es una canalización sencilla pero verificable: cada etapa recibe una entrada definida, aplica controles, genera una evidencia y deja un reporte de auditoría. Esta separación facilita localizar errores, repetir el proceso y migrar cada componente a una plataforma cloud real sin cambiar la lógica del negocio.

2 Arquitectura general

La solución sigue un flujo por capas. La fuente externa entrega productos en JSON. La ingesta almacena una copia estructurada en SQLite. La limpieza corrige calidad y crea variables analíticas. El enriquecimiento cruza catálogos multiformato mediante la categoría normalizada. GitHub Actions rodea todo el proceso como mecanismo de automatización, mientras los archivos TXT y los artefactos conservan la trazabilidad.



Figura 1. Flujo completo de la arquitectura implementada

Lectura del diagrama. La fila central representa el movimiento de los datos y la franja superior representa la automatización transversal. Cada fase entrega un archivo analítico y un reporte verificable antes de alimentar la siguiente etapa.

- Los datos avanzan de izquierda a derecha sin perder los 194 productos.
- Las auditorías y los artefactos permiten revisar el resultado de cada ejecución.

2.1 Componentes principales

Capa	Componente	Responsabilidad
Fuente	DummyJSON API	Entrega registros de productos en JSON.
Ingesta	ingestion.py	Extrae, valida, crea la tabla products y genera evidencias.
Persistencia	SQLite	Almacena los 194 productos en ingestion.db.
Limpieza	cleaning.py	Corrige tipos, nulos, rangos y agrega variables derivadas.
Enriquecimiento	enrichment.py	Integra seis formatos mediante joins por category.
Automatización	GitHub Actions	Ejecuta scripts y publica resultados como artefactos.
Trazabilidad	TXT y README	Registra conteos, operaciones, resultados e instrucciones.

2.2 Fases del procesamiento

EA1 Ingesta. El script consulta el endpoint de productos, selecciona doce campos, normaliza dos nombres y crea la tabla products. La clave primaria id evita registros duplicados. La auditoría compara los identificadores del API y SQLite antes de declarar la ejecución exitosa.

EA2 Preprocesamiento. El script lee SQLite, estandariza textos, convierte tipos numéricos, controla rangos y completa las marcas faltantes. Los precios atípicos se marcan mediante el rango intercuartílico por categoría en lugar de eliminarse. También se calcula price_after_discount.

EA3 Enriquecimiento. El dataset limpio se une con seis catálogos: segmentos, proveedores, impuestos, garantías, envíos y manejo. La operación many_to_one confirma que cada categoría tiene una sola referencia en cada catálogo. Luego se calculan price_with_tax y restock_priority.

3 Flujo de datos y automatización

Paso	Entrada	Proceso	Salida
1	API JSON	ingestion.py	SQLite + Excel + auditoría
2	ingestion.db	cleaning.py	cleaned_data.xlsx + reporte
3	Dataset limpio + 6 catálogos	enrichment.py	enriched_data.xlsx + reporte
4	Repositorio	GitHub Actions	Logs y artefactos descargables

Los workflows se activan por cambios en la carpeta correspondiente o de forma manual. Cada uno descarga el repositorio, configura Python, instala las dependencias, ejecuta el script y publica sus evidencias. El estado verde confirma que el proceso terminó sin errores; el artefacto permite revisar los archivos producidos por esa ejecución.

La separación por actividad evita que una modificación de documentación ejecute tareas innecesarias. También mantiene la trazabilidad entre código, datos y resultados, porque cada ejecución queda asociada al commit que la originó.

3.1 Simulación del entorno cloud

Servicio cloud habitual	Componente utilizado	Función simulada
Repositorio de código	GitHub	Versionamiento, colaboración y almacenamiento del proyecto.
Cómputo administrado	GitHub Actions runner	Ejecución temporal y automatizada de los scripts.
Base analítica	SQLite	Persistencia estructurada en un archivo portable.
Object storage	Archivos del repositorio y artifacts	Conservación y descarga de evidencias.
Monitoreo	Logs y reportes TXT	Seguimiento de conteos, cruces y resultados.

4 Modelo de datos

El proyecto combina un modelo físico y un modelo lógico. El modelo físico está implementado en SQLite y contiene una sola tabla llamada products. El modelo lógico aparece durante el enriquecimiento: seis catálogos aportan atributos por categoría y se relacionan con muchos productos. Estas relaciones se validan en Pandas, pero no se materializan como claves foráneas dentro de SQLite.

4.1 Esquema físico de SQLite

Campo	Tipo	PK	Descripción
id	INTEGER	Sí	Identificador único del producto.
title	TEXT	No	Nombre del producto.
description	TEXT	No	Descripción suministrada por el API.
category	TEXT	No	Categoría y clave lógica de integración.
price	REAL	No	Precio de lista.
discount_percentage	REAL	No	Porcentaje de descuento.
rating	REAL	No	Calificación entre 0 y 5.

Campo	Tipo	PK	Descripción
stock	INTEGER	No	Unidades disponibles.
brand	TEXT	No	Marca del producto.
sku	TEXT	No	Código comercial.
weight	REAL	No	Peso informado por la fuente.
availability_status	TEXT	No	Estado de disponibilidad.

La clave primaria id garantiza la identidad de cada producto. El esquema no declara claves foráneas porque el API entrega categorías como texto y los catálogos complementarios permanecen como archivos. La normalización previa de category permite usarla como clave lógica sin modificar el diseño físico de la EA1.

4.2 Diagrama del modelo lógico



Figura 2. Relación lógica entre products, los catálogos y el dataset enriquecido

Lectura del modelo. products mantiene el detalle por producto. Cada catálogo conserva una referencia única por categoría y aporta uno o dos atributos. El archivo enriquecido reúne los campos físicos, los campos derivados y las variables externas.

- Cardinalidad lógica: una categoría de referencia se asocia con varios productos.
- La integridad de la relación se valida durante el merge de Pandas.

4.3 Campos generados durante el procesamiento

Campo	Tipo	Origen	Uso analítico
price_outlier	BOOLEAN	EA2	Indica precio atípico dentro de su categoría.
price_after_discount	REAL	EA2	Precio después de aplicar el descuento.
segment	TEXT	JSON	Segmento comercial de la categoría.
supplier_region	TEXT	XLSX	Origen nacional o importado del proveedor.
replenishment_days	INTEGER	XLSX	Tiempo estimado de reposición.
tax_rate	REAL	CSV	Tasa utilizada para calcular el precio con impuesto.
warranty_months	INTEGER	XML	Meses de garantía por categoría.
shipping_class	TEXT	HTML	Tratamiento requerido para el envío.
handling_note	TEXT	TXT	Recomendación operativa de manejo.
price_with_tax	REAL	EA3	Precio descontado más impuesto.
restock_priority	BOOLEAN	EA3	Alerta por poco stock y reposición lenta.

4.4 Relaciones y reglas de integridad

- products.id es único y obligatorio; funciona como clave primaria física.
- Cada catálogo contiene una sola fila por category; la relación lógica es uno a muchos respecto a products.
- El merge many_to_one detiene la ejecución si una fuente presenta claves repetidas incompatibles.
- Los joins son de tipo left para conservar todos los productos del dataset base.
- La auditoría cuenta coincidencias y registros no relacionados para cada fuente.
- El dataset final conserva 194 filas y no presenta nulos en los atributos agregados.

4.5 Justificación del modelo

La tabla products mantiene el nivel de detalle más útil: una fila por producto. Los catálogos se administran por categoría porque sus atributos se comparten entre varios productos. Esta decisión evita repetir reglas de negocio en cada archivo fuente y hace que la integración sea verificable mediante relaciones N:1.

El resultado final es desnormalizado porque está destinado al análisis. Tener precio, inventario, proveedor, impuesto, garantía y condiciones logísticas en una misma fila reduce los cruces que necesitaría una herramienta de visualización. Al mismo tiempo, conservar las fuentes separadas permite rastrear el origen de cada campo.

5 Justificación de herramientas y tecnologías

Herramienta	Aporte	Decisión en el proyecto	Escalabilidad
SQLite	Base portable, SQL y sin servidor separado.	Adecuado para 194 registros y una simulación local.	Migrar a PostgreSQL, BigQuery o Snowflake cuando aumenten usuarios y concurrencia.
Pandas	Lectura multiformato, limpieza y joins en Python.	Reduce la complejidad y permite evidencias rápidas.	Procesa principalmente en memoria; no es distribuido.
PySpark	Alternativa distribuida para grandes volúmenes.	No se implementó porque el tamaño actual no lo justifica.	Recomendado al migrar a un clúster o lago de datos.
GitHub Actions	Automatiza entornos, comandos y artefactos.	Aporta repetibilidad y trazabilidad por commit.	Los runners son temporales y no sustituyen un orquestador de producción.

5.1 Eficiencia y mantenibilidad

Los scripts separan responsabilidades y usan rutas relativas, lo que facilita ejecutar el proyecto localmente o en GitHub Actions. Los archivos requirements.txt fijan las dependencias de cada actividad. Los README explican la instalación, la ejecución y la verificación de evidencias. Esta organización permite modificar una etapa sin reescribir las demás.

La arquitectura es eficiente para el alcance académico porque evita infraestructura permanente. SQLite no necesita un servidor y GitHub Actions solo consume recursos durante una ejecución. Para un entorno real, la eficiencia deberá medirse también por volumen, latencia, frecuencia de actualización, concurrencia y costo.

6 Controles y trazabilidad

Etapas	Control principal	Evidencia
EA1	Conteo API vs. SQLite, IDs duplicados y campos obligatorios.	ingestion.txt
EA2	Nulos, duplicados, rangos, marcas imputadas y atípicos.	cleaning_report.txt
EA3	Coincidencias, diferencias, duplicados de clave y columnas agregadas.	enrichment_report.txt
CI	Código de salida, logs y existencia de archivos.	GitHub Actions + artifacts

La trazabilidad conecta cuatro elementos: el commit que contiene el código, la ejecución del workflow, los logs generados durante el proceso y los archivos publicados como artefactos. Si una ejecución falla, el log identifica el paso y evita presentar resultados parciales como exitosos.

7 Conclusiones

La arquitectura integra las tres fases del proyecto con entradas y salidas claramente identificadas. El uso de SQLite, Pandas y GitHub Actions resulta proporcional al tamaño actual y permite reproducir el proceso sin administrar servidores.

El modelo físico conserva una estructura simple y estable, mientras el modelo lógico añade información por categoría sin perder productos. La separación entre fuentes, transformaciones y evidencias favorece la auditoría y facilita el análisis del archivo enriquecido.

La principal limitación es que la solución no es una plataforma Big Data distribuida: los datos se procesan en memoria, SQLite ofrece concurrencia limitada y los archivos del repositorio no reemplazan un lago de datos. Estas decisiones son válidas para la simulación académica, pero deben evolucionar si crecen el volumen, la frecuencia o el número de usuarios.

8 Recomendaciones

- Migrar los archivos de entrada y salida a almacenamiento de objetos y usar formatos columnares como Parquet.
- Materializar los catálogos como tablas de dimensión con claves sustitutas y reglas de vigencia.
- Usar PySpark o un servicio administrado cuando el volumen supere la capacidad de memoria de una sola máquina.
- Agregar pruebas automáticas de calidad, contratos de esquema y alertas ante caídas de coincidencias.
- Separar ambientes de desarrollo y producción, administrar secretos y definir políticas de retención.
- Incorporar un catálogo de datos y metadatos de linaje para documentar el origen de cada variable.

9 Referencias

DummyJSON. (2026). Products API. <https://dummyjson.com/docs/products>

GitHub. (2026). GitHub Actions documentation. <https://docs.github.com/en/actions>

Institución Universitaria Digital de Antioquia. (2026). Símbolos institucionales. <https://www.iudigital.edu.co/index.php/nuestra-institucion/filosofia-institucional/simbolos>

Pandas Development Team. (2026). Pandas documentation. <https://pandas.pydata.org/docs/>

SQLite Consortium. (2026). About SQLite. <https://www.sqlite.org/about.html>

The Apache Software Foundation. (2026). PySpark documentation. <https://spark.apache.org/docs/latest/api/python/>